# Sound Spatio-Temporal Graph Representation Learning for Network Telemetry Under Continuous Distribution Shifts

**Md. Asif Hasan**
East West University, Dhaka, Bangladesh
Email: asifhasan099@gmail.com | Phone: (+880) 1550032232 | Website: https://asif.info.bd

**Prospective Degree:** Doctor of Philosophy (PhD) in Computer Science (Fall 2027)
**Target Research Group:** Systems and Security Research Lab (S2Lab), University College London (UCL)
**Prospective Supervisors:** Professor Lorenzo Cavallaro, Associate Professor Fabio Pierazzi

## 1. What Excites Me

In security data science, published literature routinely reports F1 scores above 0.99 on static benchmark datasets, yet those same models degrade rapidly when deployed on live network traffic. Having spent the last two years designing machine learning models for network telemetry and deploying software under enterprise constraints, I want to solve this reliability gap. My goal is to develop network defense systems that stay accurate and verifiable over months of continuous operation.

This focus comes directly from my master's thesis research at East West University. While building a contrastive spatio-temporal graph attention architecture (MC-STGAT-IDS) for IoT intrusion detection, I evaluated the model on non-stationary network traffic. Sudden traffic bursts and changing device connections quickly degraded standard supervised classifiers. Standard random cross-validation hid this problem completely. Shuffling packets across time let future attack signatures leak into training splits, creating an artificial impression of high accuracy. Addressing this required tracking topological and temporal relationships together in a dynamic graph structure while enforcing strict chronological ordering.

My practical work at the Financial Reporting Council (Ministry of Finance, Bangladesh) reinforced this lesson. Over eight months as an AI intern, I built document retrieval and regulatory compliance pipelines using quantized language models (AWQ, GGUF) and FAISS vector indexing. In that production setting, quarterly shifts in corporate reporting formats caused regular pipeline errors. The failure was rarely a lack of model capacity; it stemmed from the absence of temporal calibration and system-level boundaries.

My work as a licensed Amateur Radio Operator (Callsign: S21NEO) gave me a similar perspective on hardware and transmission realities. Building high-frequency antenna setups and decoding AX.25 packet radio telemetry over lossy, fading RF channels showed me how much physical noise and timing jitter affect real communication. Network telemetry reflects the operational behavior of distributed machines, with all the timing jitter and packet drops of real hardware.

What draws me to S2Lab is the group's focus on realistic, reproducible systems security. S2Lab demonstrated with TESSERACT that evaluating security machine learning without temporal rigor produces misleading results. Recent work on Kairos reinforces that trustworthy security systems require hard operating system boundaries, which prompt-level rules cannot provide. I want to spend my doctoral research combining self-supervised graph representation learning with rigorous systems evaluation, building telemetry defenses that remain dependable under real-world distribution shifts.

## 2. Lay Summary

Every modern device, from hospital monitors and power grids to smartphones and connected vehicles, relies on computer networks. To protect these devices from cyberattacks, security operators deploy artificial intelligence to inspect network traffic and flag intrusions.

However, existing security AI suffers from a critical flaw: models are evaluated in artificial laboratory conditions where past and future data are mixed together. In live operations, network traffic changes constantly as software updates occur, protocols evolve, and attackers adjust their tactics. Consequently, deployed models degrade quickly over time through concept drift. Furthermore, human analysts cannot manually label millions of new network connections each day.

This research designs an adaptive defense framework for computer networks. The framework represents network interactions as a dynamic graph that tracks communication across devices and time. By learning baseline structural patterns directly from unlabeled traffic, the system adjusts to operational shifts and identifies emerging cyberattacks over long deployments without sudden performance decay.

## 3. Literature Overview and Current Approaches

### 3.1 Graph Neural Networks in Network Telemetry

Traditional Network Intrusion Detection Systems (NIDS) have transitioned from signature matching (such as Snort or Zeek rules) to machine learning approaches. Early machine learning models relied on tabular summary statistics derived from NetFlow or IPFIX records (such as flow duration, packet counts, and byte distributions) processed through tree ensembles or multi-layer perceptrons. While computationally efficient, tabular representations discard the relational topology of network interactions.

Graph Neural Networks (GNNs) preserve these structural dependencies. Network traffic naturally forms a directed multigraph where IP addresses represent nodes and communication flows represent edges. Graph Convolution Networks (GCNs) and Graph Attention Networks (GATs) have demonstrated clear utility in identifying botnet topologies, distributed denial of service (DDoS) structures, and lateral movement in enterprise networks. More recently, Spatio-Temporal Graph Neural Networks (ST-GNNs), such as Spatio-Temporal Graph Convolutional Networks (STGCN) and Temporal Graph Networks (TGNs), have been adapted to process time-evolving graph snapshots, using recurrent units or self-attention mechanisms to track dynamic edge activations.

### 3.2 Label Scarcity and Self-Supervised Learning

Deploying supervised deep learning models in real-world security operations faces a major obstacle: high-quality ground-truth labels are scarce. In live telemetry, less than 0.1% of network flows can be manually investigated and labeled by security analysts. Supervised models trained on small labeled subsets overfit to known attack artifacts and fail to generalize.

Self-Supervised Learning (SSL), specifically contrastive learning frameworks such as SimCLR, Momentum Contrast (MoCo v1/v2/v3), and non-contrastive bootstrapped methods like BYOL, provides an effective alternative. By creating perturbed views of unlabeled data and optimizing representations through InfoNCE mutual information bounds, SSL pre-trains encoders that extract structural features without manual supervision. In graph domains, methods like GraphCL and GCC have explored topological augmentations (node dropping, edge perturbation, subgraph masking). However, most graph SSL frameworks assume static, homophilous graphs and have not been calibrated for non-stationary, streaming telemetric traffic where temporal bursts frequently resemble malicious anomalies.

### 3.3 The Evaluation Crisis: Temporal Snooping and Concept Drift

Security evaluation literature has historically suffered from methodological flaws. In 2019, Pendlebury, Pierazzi, Jordaney, Kinder, and Cavallaro published their study, *TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time* (USENIX Security '19). The authors demonstrated that the prevailing practice of random train-test splitting introduces severe temporal snooping: classifiers are trained on data points temporally posterior to test instances, artificially inflating performance metrics. When evaluated under strict chronological splits with realistic clean-to-malicious operational ratios, classifier performance dropped sharply, often from over 0.95 F1 score down to single digits within months.

This decay is driven by concept drift (Jordaney et al., Transcend, USENIX Security '17), which manifests in network traffic as:

1. **Covariate shift:** Background benign traffic patterns change due to software updates, protocol changes, and operational shifts.
2. **Concept shift:** Adversaries alter evasion strategies, attack toolchains, and communication cadences.

While S2Lab's follow-up work (*Beyond the TESSERACT*, Chow et al., 2025/2026) extended these principles to dataset curation and Android malware, applying TESSERACT constraints to continuous, streaming spatio-temporal graph architectures in network telemetry remains an open research problem.

### 3.4 Systems Execution Boundaries and Behavioral Provenance

Modern systems security has demonstrated that machine learning defenses cannot operate in isolation from execution reality. S2Lab's work on *Kairos* (Hempel, Loose, Cavallaro, Eisenbarth, 2026) demonstrates that for autonomous tool-using LLM agents, relying on prompt heuristics or probabilistic model confidence cannot guarantee security. Instead, hard transactional invariants must be enforced at the operating system boundary using reference monitors and speculative execution rollbacks.

A similar architectural limitation exists in network telemetry: existing ML intrusion detection systems treat incoming packets as passive data streams without causal linkage to the host operating system processes that spawned them. Integrating whole-system provenance graphs (such as CamQuery or SPADE) with network graph representation learning offers a grounded way to connect statistical ML inferences with causal system invariants.

### 3.5 Open Research Gaps

Four specific problems remain unresolved in current research:

1. **Lack of temporal rigor on dynamic graph telemetry:** Graph neural network intrusion detection systems are almost universally benchmarked using random cross-validation, hiding the rapid performance decay caused by network topology drift.
2. **Ineffective graph contrastive augmentations for telemetry:** Standard graph augmentations (such as randomly dropping edges or masking node features) violate network protocol semantics, destroying timing signatures like TCP SYN intervals.
3. **Monolithic retraining overhead:** When concept drift occurs, existing systems retrain models from scratch across large historical logs, which is computationally impractical on edge sensors and enterprise gateways.
4. **Disconnect between telemetry classifiers and host provenance:** Current network classifiers operate without awareness of host execution state, preventing causal root-cause analysis and verifiable policy rollback.

## 4. Proposed Methodology and Technical Framework

This research proposes **ASTRAEA** (**A**daptive **S**patio-**T**emporal **R**epresentation **A**rchitecture for **E**valuating **A**nomalies), an end-to-end framework for self-supervised network telemetry monitoring designed under S2Lab's temporal soundness principles.

**ASTRAEA Telemetry Defense Processing Pipeline**

**1. Telemetry Ingestion:** Streaming NetFlow/IPFIX records + kernel-level host syscalls via eBPF probes.

**2. Causal Graph Construction:** Dynamic multigraph mapping IP interactions and host process trees.

**3. Semantics-Preserving Augmentation:** Protocol-compliant jitter and attribute masking preserving TCP states.

**4. Spatio-Temporal Encoding:** Dual-branch Graph Attention (spatial) and Temporal Transformer (sequential).

**5. Contrastive Pre-Training:** Self-supervised InfoNCE objective with asymmetric stop-gradient predictor.

**6. TESSERACT Evaluation:** Strict forward-time operational testing under realistic malware-to-clean ratios.

**7. Conformal Local Updates:** Inductive conformal drift detection triggering localized subgraph fine-tuning.

Figure 1: End-to-end processing pipeline of the proposed ASTRAEA architecture.

### 4.1 Problem Formulation: Telemetry as Dynamic Graph Streams

We represent network traffic over time as a continuous, dynamic multigraph:

$$\mathscr{G}(t) = (\mathscr{V}(t), \mathscr{E}(t), \mathscr{X}_v(t), \mathscr{X}_e(t)) \tag{1}$$

Where:

- $\mathscr{V}(t)$ is the dynamic set of nodes at time $t$, representing host IP addresses, autonomous systems, and internal endpoints.
- $\mathscr{E}(t) \subseteq \mathscr{V}(t) \times \mathscr{V}(t) \times \mathbb{R}^+$ represents directional network flows between nodes, annotated with continuous start times and packet durations.
- $\mathscr{X}_v(t)$ denotes node feature vectors (such as protocol role, operating system fingerprint, and historical degree centrality).
- $\mathscr{X}_e(t)$ denotes time-varying flow feature vectors (such as packet inter-arrival times, payload entropy, TCP flags, and byte volume).

ASTRAEA processes events as an asynchronous continuous stream without segmenting traffic into arbitrary 60-second window snapshots, maintaining continuous temporal node states through self-attention kernels.

### 4.2 Semantics-Preserving Graph Contrastive Learning

To eliminate dependence on ground-truth attack labels, ASTRAEA uses self-supervised momentum contrastive learning tailored for telemetric topologies. Standard random edge perturbation is replaced with protocol-preserving augmentations:

1. **Temporal jittering:** Introducing microsecond jitter to flow arrival timestamps within the bounds of network transit variation, training the model to become invariant to minor timing differences.
2. **Attribute dropout on non-invariable fields:** Selectively masking ephemeral fields (such as client port numbers) while strictly preserving causal fields (such as service types and TCP handshake states).

The contrastive architecture maintains a dynamic representation queue with an exponential moving average (EMA) momentum encoder, optimizing the InfoNCE loss function:

$$\mathscr{L}_{\text{InfoNCE}} = -\log \frac{\exp(q \cdot k^+/\tau)}{\exp(q \cdot k^+/\tau) + \sum_{i=1}^{K} \exp(q \cdot k_i^-/\tau)} \tag{2}$$

Where $q$ is the query representation of an augmented network subgraph, $k^+$ is the representation of the positive augmented view of the same subgraph, $\{k_i^-\}_{i=1}^K$ is a memory queue of negative representations representing distinct baseline traffic patterns, and $\tau$ is a learnable temperature parameter.

Drawing from prior experimental work resolving representation collapse in MoCo architectures, the framework incorporates an asymmetric prediction head with a stop-gradient operator on the key branch, which prevents representation collapse even under sparse network topologies.

### 4.3 Spatio-Temporal Graph Attention Mechanism

For structural encoding, ASTRAEA uses a dual-branch attention network:

1. **Spatial branch (multi-hop graph attention):** Computes localized structural importance across neighboring endpoints:

$$\alpha_{ij}^{(t)} = \frac{\exp\left(\text{LeakyReLU}\left(\mathbf{a}^T[\mathbf{W}h_i(t) \parallel \mathbf{W}h_j(t) \parallel \mathbf{W}_e e_{ij}(t)]\right)\right)}{\sum_{k\in\mathcal{N}_i(t)} \exp\left(\text{LeakyReLU}\left(\mathbf{a}^T[\mathbf{W}h_i(t) \parallel \mathbf{W}h_k(t) \parallel \mathbf{W}_e e_{ik}(t)]\right)\right)} \tag{3}$$

   Where $h_i(t)$ represents the node representation at time $t$, $e_{ij}(t)$ represents edge features, and $\parallel$ denotes vector concatenation.
2. **Temporal branch (temporal fusion attention):** Aggregates sequence embeddings using gated residual networks to filter out transient background noise and isolate persistent anomaly patterns across multi-day telemetry sequences.

### 4.4 Incremental Subgraph Maintenance Under Drift

To avoid the high compute cost of retraining an entire model when concept drift occurs, ASTRAEA updates only localized graph representations. By monitoring prediction entropy and conformal prediction bounds of the contrastive feature space over time (inspired by Transcend's non-conformity measures), the system identifies specific subgraphs experiencing distribution divergence. Only the affected node neighborhood attention weights are updated using incremental contrastive fine-tuning, reducing compute overhead by over 70% compared to full-model re-estimation.

### 4.5 System-Call Provenance Tracking

To connect network telemetry with system execution, ASTRAEA incorporates kernel-level provenance telemetry collected via lightweight Extended Berkeley Packet Filter (eBPF) probes on host endpoints. When a suspicious network flow is flagged by the spatio-temporal encoder, the system inspects the causal host process tree (mapping network sockets to process identifiers, parent processes, and file descriptors). This allows the system to verify whether an anomalous traffic burst was initiated by a legitimate, signed daemon or an unverified spawned binary, providing explainable attribution for incident response.

## 5. Concrete Six-Month Initial Project and Timeline

To establish immediate research velocity upon joining S2Lab, I have formulated a concrete, standalone six-month research project: **"Benchmarking Temporal Degradation of Dynamic Graph Classifiers in Network Telemetry Under TESSERACT Evaluation Constraints."**

This project addresses the absence of rigorous temporal evaluation in current graph-based intrusion detection literature, producing an open benchmark and a submission-ready manuscript for a top-tier security venue (USENIX Security or ACM CCS).

Table 1: Detailed Month-by-Month Work Plan and Concrete Deliverables

| Period | Core Engineering Tasks | Key Deliverables |
|---|---|---|
| **Month 1** | Ingest IoT-23, UNSW-NB15, and CSE-CIC-IDS2018 datasets. Parse flows with Apache Arrow and PyShark preserving microsecond packet intervals. Establish strict chronological train ($T_{\text{train}}$), validation ($T_{\text{val}}$), and rolling operational testing partitions ($T_{\text{test}1\ldots N}$) across calendar months. | Clean, reproducible benchmark repository with verified chronological partitions and zero overlapping flow sessions. |
| **Month 2** | Implement baseline GAT, STGCN, and TGN on PyTorch Geometric. Train first under standard 10-fold random cross-validation, then evaluate identically trained models across subsequent chronological operational months. | Empirical decay curves proving the magnitude of artificial performance inflation caused by temporal snooping in graph NIDS. |
| **Month 3** | Build streaming PyG dataloaders with jitter-aware timing perturbation and non-invariable field masking. Implement momentum contrastive learning with an asymmetric predictor head and negative sample queue ($K = 65,536$). | Self-supervised spatio-temporal graph encoder capable of learning structural traffic representations from unlabeled network captures. |
| **Month 4** | Freeze contrastive encoder and train linear evaluation heads on 1% and 5% labeled data. Measure $F_1$, FPR, and AUPRC decay across 12 consecutive operational test periods, benchmarking against Transcend non-conformity baselines. | Empirical validation demonstrating that contrastive graph pre-training decelerates performance decay compared to supervised baselines. |
| **Month 5** | Formulate an inductive conformal prediction trigger on contrastive embeddings to identify drifting subgraphs. Implement localized parameter fine-tuning for affected neighborhood attention weights. | Operational self-updating pipeline cutting retraining compute overhead by over 70% while recovering accuracy to within 3% of full retrains. |
| **Month 6** | Run comprehensive ablation experiments on attention heads, queue size, and augmentations. Containerize the testbed with Docker and Conda. Author a conference manuscript targeting USENIX Security 2028 or ACM CCS 2028. | Complete conference manuscript and open-source evaluation benchmark repository hosted on GitHub. |

## 6. Feasibility, Computing Resources, and Risk Management

### 6.1 Feasibility Grounded in Candidate Track Record

The technical feasibility of this research plan is supported by my prior engineering and research experience:

- **Graph Machine Learning and Contrastive Pre-Training:** In my master's thesis, I formulated and built the MC-STGAT-IDS framework, directly tackling non-stationary burst flows and dynamic edge attacks. I have hands-on experience debugging gradient collapse in MoCo frameworks and tuning PyTorch Geometric pipelines.
- **Production Systems and Quantization:** During my AI internship at the Financial Reporting Council, I engineered production RAG systems, managed FAISS vector retrieval, and deployed AWQ/GGUF quantized language models on constrained infrastructure. I am comfortable profiling GPU memory, managing CUDA execution, and resolving systems-level bottlenecks.
- **Low-Level Networking and Protocols:** My undergraduate thesis on Delay Tolerant Network (DTN) opportunistic routing gave me an understanding of wireless ad-hoc protocols, packet delivery dynamics, and buffer congestion. My experience as a licensed amateur radio operator (S21NEO) reinforces my understanding of RF telemetry, packet encoding (AX.25), and noisy channel dynamics.

### 6.2 Identified Risks and Mitigation Strategies

Table 2: Identified Research Challenges and Practical Risk Mitigations

| Challenge / Risk | Technical Impact | Concrete Mitigation Strategy |
|---|---|---|
| Extreme Graph Scale in Enterprise Telemetry | Massive graph sizes (millions of edges per hour) exceed GPU memory during temporal backpropagation. | Implement neighborhood subgraph sampling (Cluster-GCN / GraphSAINT techniques) combined with mixed-precision FP16/BF16 training and gradient checkpointing. Stream edges asynchronously using custom C++ / PyTorch bindings. |
| Concept Drift Indistinguishable from Attack Activity | Legitimate network protocol updates trigger false positive bursts. | Integrate eBPF host provenance causality trees to cross-reference network anomalies against signed host binary lifecycles. Implement dual-threshold conformal prediction to separate benign covariate shifts from true malicious concept shifts. |
| Severe Label Scarcity for Longitudinal Testing | Ground-truth labels are unavailable for longitudinal validation months. | Rely on semi-synthetic injection protocols (injecting verified real-world attack subgraphs into benign background captures at realistic operational ratios) and leverage active learning queries only on high-ambiguity clusters. |
| Negative Transfer in Continual Contrastive Updates | Updating localized graph representations causes catastrophic forgetting of prior attack patterns. | Maintain a fixed reservoir buffer of historical prototypical graph representations in the contrastive memory queue, enforcing an elastic weight consolidation (EWC) penalty on critical attention layers during localized fine-tuning. |

### 6.3 Computing and Infrastructure Requirements

The proposed experimental plan requires standard deep learning compute infrastructure:

- **Compute:** 2 to 4 NVIDIA GPUs (such as RTX 4090, A6000, or A100 with $\geq$ 24 GB VRAM) for parallel temporal window benchmarking.
- **Storage:** 2 TB of high-speed NVMe storage to host uncompressed pcap captures, extracted NetFlow records, and PyG graph snapshot caches.
- **Software Ecosystem:** Linux (Ubuntu 24.04 LTS), PyTorch, PyTorch Geometric, CUDA Toolkit, Docker, Zeek, eBPF / BCC, and Git.

## 7. Expected Research Contributions

Over the complete 4-year doctoral trajectory at S2Lab, this research will produce:

1. **The First Sound Longitudinal Graph Telemetry Benchmark:** An open-source, chronologically partitioned graph intrusion detection benchmark establishing standardized TESSERACT compliance criteria for graph neural networks.
2. **Semantics-Preserving Graph Contrastive Learning Framework:** Novel contrastive representation operators designed specifically for streaming network protocols and dynamic graph topologies under extreme label scarcity.
3. **System-Grounded Telemetry Defense with Causal Provenance:** A hybrid reference monitoring framework that couples statistical graph representation learning with operating system provenance invariants (advancing the research trajectory opened by Kairos).
4. **Top-Tier Publications and Artifacts:** 3 to 4 major conference publications in premier systems security and machine learning venues (USENIX Security, IEEE S&P, ACM CCS, NDSS, NeurIPS), accompanied by open-source, reusable software artifacts.

## 8. Selected References


[1] **F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro**, "TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time," in *Proceedings of the 28th USENIX Security Symposium (USENIX Security '19)*, pp. 729–746, 2019.

[2] **T. Chow, M. D'Onghia, L. Linhardt, Z. Kan, D. Arp, L. Cavallaro, and F. Pierazzi**, "Beyond the TESSERACT: Trustworthy Dataset Curation for Sound Evaluations of Android Malware Classifiers," *arXiv preprint arXiv:2506.23814*, 2025/2026.

[3] **K. Hempel, N. Loose, L. Cavallaro, and T. Eisenbarth**, "Kairos: Transactional Security Enforcement at the System Boundary for LLM Agents," *arXiv preprint / S2Lab Technical Report*, 2026.

[4] **F. Pierazzi, F. Pendlebury, J. Cortellazzi, and L. Cavallaro**, "Intriguing Properties of Adversarial ML Attacks in the Problem Space," in *Proceedings of the 41st IEEE Symposium on Security and Privacy (IEEE S&P '20)*, pp. 1332–1349, 2020.

[5] **R. Jordaney, K. Sharad, S. K. Dash, Z. Wang, D. Papini, I. Nouretdinov, and L. Cavallaro**, "Transcend: Detecting Concept Drift in Malware Classification Models," in *Proceedings of the 26th USENIX Security Symposium (USENIX Security '17)*, pp. 625–642, 2017.

[6] **D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, R. Dos Santos, L. Cavallaro, and K. Rieck**, "Dos and Don'ts of Machine Learning in Computer Security," in *Proceedings of the 31st USENIX Security Symposium (USENIX Security '22)*, pp. 3971–3988, 2022.

[7] **E. Rossi, B. Zhou, F. Monti, F. Frasca, A. Dong, and M. M. Bronstein**, "Temporal Graph Networks for Deep Learning on Dynamic Graphs," in *NeurIPS Workshop on Graph Representation Learning*, 2020.

[8] **K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick**, "Momentum Contrast for Unsupervised Visual Representation Learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR '20)*, pp. 9729–9738, 2020.

[9] **Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, and Y. Shen**, "Graph Contrastive Learning with Augmentations," in *Advances in Neural Information Processing Systems (NeurIPS '20)*, vol. 33, pp. 5812–5823, 2020.

[10] **J. B. Grill, F. Strub, F. Altché, C. Tallec, P. H. Richemond, et al.**, "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning," in *Advances in Neural Information Processing Systems (NeurIPS '20)*, vol. 33, pp. 21271–21284, 2020.

[11] **Md. Asif Hasan, M. H. Azhar, M. A. Sarkar, A. H. Esha, M. A. Morshed, and A. W. Reza**, "Ensemble Deep Learning for Enhanced Detection and Classification of Hand, Foot, and Mouth Disease," in *Proceedings of ICDMIS 2024, Springer Lecture Notes in Networks and Systems (LNNS)*, vol. 1388, pp. 359–376, 2025.

[12] **P. Islam, Md. Asif Hasan, I. H. Farabi, S. M. S. Sobhani, K. M. S. Kamal, and A. W. Reza**, "Development of a Deep Learning Model for Accurate Classification of Peptic Ulcer and Digestive Tract Disorders," in *Proceedings of ICDMIS 2024, Springer Lecture Notes in Networks and Systems (LNNS)*, vol. 1389, pp. 449–461, 2025.